LAPORAN PRAKTIKJUM INFORMATION SYSTEM
MODUL 6
TECHNIQUE OF PERFORMANCE OF SIMILARITY TO IDEAL SOLUTION (TOPSIS)

Oleh :

Moh. Azka Fariqi (240605110257)

Kelas: C

Dosen Pengampu :

ASHRI SHABRINA AFRAH, M.T.

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM

MALANG

2026

1. Tujuan Penelitian

Praktikum ini bertujuan untuk:

1.1 Memahami konsep dasar metode Technique for Order Preference by Similarity to Ideal Solution (TOPSIS) dalam Sistem Pendukung Keputusan.

1.2 Menentukan nilai matriks keputusan berdasarkan alternatif dan kriteria yang telah ditetapkan.

1.3 Menghitung matriks keputusan ternormalisasi dan matriks normalisasi terbobot.

1.4 Menentukan solusi ideal positif dan solusi ideal negatif berdasarkan jenis atribut *benefit* dan *cost*.

1.5 Menghitung jarak setiap alternatif terhadap solusi ideal positif dan negatif.

1.6 Menghitung nilai preferensi setiap alternatif serta menentukan peringkat alternatif terbaik.

1.7 Mengimplementasikan metode TOPSIS menggunakan Microsoft Excel dan program Python.

2. Dasar Teori

Technique for Order Preference by Similarity to Ideal Solution (TOPSIS) merupakan salah satu metode yang digunakan dalam Sistem Pendukung Keputusan untuk menentukan alternatif terbaik berdasarkan sejumlah kriteria. Prinsip utama TOPSIS adalah alternatif yang terbaik harus mempunyai jarak paling dekat terhadap solusi ideal positif dan sekaligus mempunyai jarak paling jauh terhadap solusi ideal negatif. Untuk mengukur kedekatan masing-masing alternatif terhadap kedua solusi tersebut, TOPSIS menggunakan perhitungan jarak Euclidean.

Dalam proses perhitungannya, TOPSIS dimulai dengan membentuk matriks keputusan yang berisi nilai setiap alternatif terhadap masing-masing kriteria. Matriks tersebut kemudian dinormalisasi agar nilai dari kriteria yang mempunyai skala berbeda dapat dibandingkan. Nilai hasil normalisasi selanjutnya dikalikan dengan bobot masing-masing kriteria sehingga diperoleh matriks keputusan ternormalisasi terbobot. Setelah itu ditentukan solusi ideal positif dan solusi ideal negatif. Pada atribut benefit, solusi ideal positif menggunakan nilai maksimum dan solusi ideal negatif menggunakan nilai minimum. Sebaliknya, pada atribut cost, solusi ideal positif menggunakan nilai minimum dan solusi ideal negatif menggunakan nilai maksimum.

Tahap berikutnya adalah menghitung jarak masing-masing alternatif terhadap solusi ideal positif dan solusi ideal negatif. Nilai tersebut digunakan untuk menghitung nilai preferensi V_i. Alternatif dengan nilai preferensi terbesar merupakan alternatif yang paling

dekat dengan kondisi ideal dan dipilih sebagai alternatif terbaik. Dalam studi kasus pada modul, metode TOPSIS digunakan untuk menentukan calon debitur berdasarkan kriteria pendapatan rutin, status kepemilikan jaminan, dan nilai jaminan.

3. Latihan Praktikum

3.1 Membuat Tabel Kriteria dan Bobot

	A	B	C	D
1				
2				
3		1. Kriteria dan Bobot Kriteria		
4		Kriteria	Bobot Kriteria	
5		Pendapatan Rutin (C1)	50	
6		Status Kepemilikan Jaminan (C2)	20	
7		Nilai Jaminan (C3)	30	
8				

Pada tahap ini membuat tabel kriteria dan bobot kriteria sesuai dengan tabel yang dicontohkan pada modul

3.2 Membuat Tabel Pembobotan Skor untuk tiap Kriteria

	A	B	C	D
10				
11		1b.Pembobotan Skor untuk Kriteria C1		
12		**Subkriteria**	**Skor**	
13		Nilai angsuran kurang dari atau sama dengan 30% dari pendapatan rutin per bulan.	5	
14		Nilai angsuran lebih dari 30% dari pendapatan rutin per	2	
15				
16		1c. Pembobot Skor untuk Kriteria C2		
17		**Subkriteria**	**Skor**	
18		Milik sendiri	5	
19		Milik orang lain	2	
20				
21		1d. Pembobotan Skor untuk Kriteria C3		
22		**Subkriteria**	**Skor**	
23		Nilai angsuran kurang dari atau sama dengan 50% nilai jual jaminan.	5	
24		Nilai angsuran lebih dari 50% nilai jual jaminan.	3	
25				

Pada tahap ini melakukan pembuatan tabel untuk penentuan bobot skor untuk tiap kriteria (C1,C2, dan C3) sesuai dengan tabel yang diberikan pada modul.

3.3 Membuat Matrik Keputusan

2. Membuat Matriks Keputusan

Alternatif	C1	C2	C3
Taufan	5	5	3
Bunga	5	2	3
Yoga	2	5	5
Desi	5	2	5

Pada tahap ini dibuat matriks keputusan berdasarkan skor setiap alternatif terhadap masing-masing kriteria. Matriks keputusan digunakan sebagai data awal untuk seluruh proses perhitungan TOPSIS.

Secara umum, matriks keputusan dapat dituliskan sebagai:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}$$

dengan:

$$x_{ij}$$

merupakan nilai alternatif ke-i pada kriteria ke-j, m adalah jumlah alternatif, dan n adalah jumlah kriteria.

3.4 Menghitung Matriks Keputusan Ternormalisasi dari masing-masing alternatif

3. Menghitung Matriks Keputusan Ternormalisasi

Alternatif	C1	C2	C3
Taufan	0,56254395	0,656532164	0,363803438
Bunga	0,56254395	0,262612866	0,363803438
Yoga	0,22501758	0,656532164	0,606339063
Desi	0,56254395	0,262612866	0,606339063

Tahap selanjutnya adalah melakukan normalisasi matriks keputusan. Normalisasi dilakukan agar nilai dari setiap kriteria berada pada skala yang sebanding sehingga dapat dibandingkan secara adil.

Rumus normalisasi TOPSIS adalah:

$$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{i=1}^{m} x_{ij}^2}}$$

Keterangan:

r_{ij} = nilai ternormalisasi alternatif ke-i pada kriteria ke-j

x_{ij}= nilai awal alternatif ke-i pada kriteria ke-j

m = jumlah alternatif

Jika nilai C1 berada di G5:G8 dan hasil normalisasi Taufan berada di G13, rumus dapat ditulis:

=G5/SQRT(G5^2+G6^2+G7^2+G8^2)

Atau versi yang lebih ringkas:

=G5/SQRT(SUMSQ(G$5:G$8))

3.5 Menghitung Matriks Keputusan Ternormalisasi Terbobot dari masing-masing alternatif

	E	F	G	H	I	J
19		4. Matrik Keputusan Ternormalisasi Terbobot				
20		**Alternatif**	**C1**	**C2**	**C3**	
21		Taufan	28,12719752	13,13064329	10,91410313	
22		Bunga	28,12719752	5,252257314	10,91410313	
23		Yoga	11,25087901	13,13064329	18,19017188	
24		Desi	28,12719752	5,252257314	18,19017188	
25						

Setelah memperoleh matriks ternormalisasi, langkah berikutnya adalah mengalikan setiap nilai normalisasi dengan bobot kriterianya.

Rumus yang digunakan:

$$y_{ij} = w_j \times r_{ij}$$

Keterangan:

y_{ij} = nilai ternormalisasi terbobot

w_j = bobot kriteria ke-j

r_{ij} = nilai ternormalisasi

Jika nilai normalisasi Taufan pada C1 berada di G13 dan bobot C1 berada di C5:

=G13*C5

Sehingga didapatkan nilai untuk Taufan G21 yaitu 28,127. Perhitungan ini kemudian diimplementasikan di cell yang lainnya

3.6 Menentukan Matriks Solusi Ideal Positif dan Negatif

	K	L	M	N	O
1					
2					
3		5. Solusi Ideal Positif dan Negatif			
4			Positif	Negatif	
5		C1	28,1272	11,25088	
6		C2	13,13064	5,252257	
7		C3	18,19017	10,9141	
8					

Pada tahap ini ditentukan solusi ideal positif dan solusi ideal negatif. Solusi ideal positif menunjukkan kondisi terbaik, sedangkan solusi ideal negatif menunjukkan kondisi terburuk.

Untuk atribut **benefit**:

$$y_j^+ = \max(y_{ij})$$

$$y_j^- = \min(y_{ij})$$

Sedangkan untuk atribut **cost** berlaku sebaliknya:

$$y_j^+ = \min(y_{ij})$$

$$y_j^- = \max(y_{ij})$$

Berikut untuk implementasi di excel:

Rumus Excel C1:

Positif:

=MAX(G21:G24)

Negatif:

=MIN(G21:G24)

3.7 Menghitung Jarak Tiap Alternatif dari Solusi Ideal Positif dan Negatif

	K	L	M	N	O	P	Q
10		6. Jarak Tiap Alternatif dari Solusi Ideal Positif dan Negatif					
11			D_i^+	D_i^-			
12		**Taufan**	7,276069	18,62469			
13		Bunga	10,72428	16,87632			
14		Yoga	16,87632	10,72428			
15		Desi	7,878386	18,37801			
16							

Setelah solusi ideal diperoleh, langkah berikutnya adalah menghitung jarak setiap alternatif terhadap solusi ideal positif dan solusi ideal negatif menggunakan jarak Euclidean.

Jarak terhadap solusi ideal positif:

$$D_i^+ = \sqrt{\sum_{j=1}^{n} \left(y_j^+ - y_{ij}\right)^2}$$

Jarak terhadap solusi ideal negatif:

$$D_i^- = \sqrt{\sum_{j=1}^{n} \left(y_{ij} - y_j^-\right)^2}$$

Rumus Excel Jarak Positif

Jika solusi ideal positif berada di M5:M7 dan data Taufan di G21:I21:

=SQRT((M5-G21)^2+(M6-H21)^2+(M7-I21)^2)

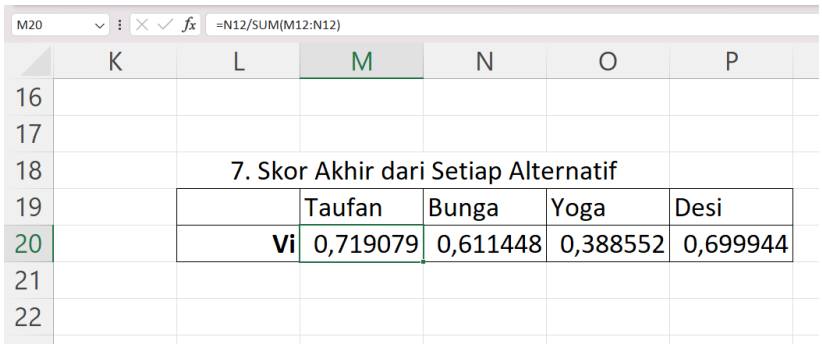
Rumus Excel Jarak Negatif

Jika solusi ideal negatif berada di N5:N7:

=SQRT((G21-N5)^2+(H21-N6)^2+(I21-N7)^2)

Secara konsep, alternatif yang baik memiliki nilai D_i^+ yang kecil dan nilai D_i^- yang besar.

3.8 Menghitung Skor Akhir Setiap Alternatif



Tahap terakhir adalah menghitung nilai preferensi setiap alternatif. Nilai preferensi digunakan untuk menentukan ranking alternatif.

Rumus:

$$V_i = \frac{D_i^-}{D_i^+ + D_i^-}$$

Keterangan:

V_i = nilai preferensi alternatif ke-i

D_i^+ = jarak terhadap solusi ideal positif

D_i^- = jarak terhadap solusi ideal negative

Rumus tersebut juga digunakan pada modul.

Rumus Excel

Jika D_i^+berada di M12 dan D_i^- berada di N12:

=N12/(M12+N12)

Atau bisa juga:

=N12/SUM(M12:N12)

Berdasarkan nilai preferensi tersebut, urutan alternatif adalah:

1. **Taufan = 0,719079**
2. **Desi = 0,699944**
3. **Bunga = 0,611448**
4. **Yoga = 0,388552**

Nilai preferensi terbesar dimiliki oleh **Taufan**, sehingga Taufan menjadi alternatif terbaik pada studi kasus ini. Modul juga menunjukkan Taufan sebagai calon debitur yang memperoleh hasil terbaik

4. Tugas Praktikum

4.1 Membuat program dengan menggunakan studi kasus pada bagian B. Praktikum. Jalankan program dan screenshoot output yang dihasilkan!

```
[ ]
import numpy as np
import pandas as pd

# Data alternatif
alternatif = ["Taufan", "Bunga", "Yoga", "Desi"]

# Matriks keputusan
X = np.array([
    [5, 5, 3],
    [5, 2, 3],
    [2, 5, 5],
    [5, 2, 5]
], dtype=float)

# Bobot kriteria
bobot = np.array([50, 20, 30], dtype=float)

# Menampilkan matriks keputusan
df_x = pd.DataFrame(
    X,
    index=alternatif,
    columns=["C1", "C2", "C3"]
)

print("=== Matriks Keputusan ===")
print(df_x)
```

```
# 1. Normalisasi matriks keputusan
normalisasi = X / np.sqrt(np.sum(X**2, axis=0))

df_normalisasi = pd.DataFrame(
    normalisasi,
    index=alternatif,
    columns=["C1", "C2", "C3"]
)

print("\n=== Matriks Keputusan Ternormalisasi ===")
print(df_normalisasi.round(6))

# 2. Matriks normalisasi terbobot
terbobot = normalisasi * bobot

df_terbobot = pd.DataFrame(
    terbobot,
    index=alternatif,
    columns=["C1", "C2", "C3"]
)

print("\n=== Matriks Normalisasi Terbobot ===")
print(df_terbobot.round(6))

# 3. Solusi ideal positif dan negatif
# Semua kriteria merupakan Benefit
ideal_positif = np.max(terbobot, axis=0)
ideal_negatif = np. min(terbobot, axis=0)

print("\n=== Solusi Ideal ===")
print("Ideal Positif :", np.round(ideal_positif, 6))
print("Ideal Negatif :", np.round(ideal_negatif, 6))
```

```python
# 4. Menghitung jarak terhadap solusi ideal
D_plus = np.sqrt(np.sum((terbobot - ideal_positif)**2, axis=1))
D_minus = np.sqrt(np.sum((terbobot - ideal_negatif)**2, axis=1))

# 5. Menghitung nilai preferensi
V = D_minus / (D_plus + D_minus)

# Menampilkan hasil
hasil = pd.DataFrame({
    "Alternatif": alternatif,
    "D+": D_plus,
    "D-": D_minus,
    "Nilai Preferensi": V
})

hasil["Ranking"] = hasil["Nilai Preferensi"].rank(
    ascending=False,
    method="min"
).astype(int)

hasil = hasil.sort_values("Ranking")

print("\n=== Hasil Akhir TOPSIS ===")
print(hasil.round(6).to_string(index=False))
```

Output:

```
=== Matriks Keputusan ===
         C1   C2   C3
Taufan  5.0  5.0  3.0
Bunga   5.0  2.0  3.0
Yoga    2.0  5.0  5.0
Desi    5.0  2.0  5.0

=== Matriks Keputusan Ternormalisasi ===
              C1        C2        C3
Taufan  0.562544  0.656532  0.363803
Bunga   0.562544  0.262613  0.363803
Yoga    0.225018  0.656532  0.606339
Desi    0.562544  0.262613  0.606339

=== Matriks Normalisasi Terbobot ===
               C1         C2         C3
Taufan  28.127198  13.130643  10.914103
Bunga   28.127198   5.252257  10.914103
Yoga    11.250879  13.130643  18.190172
Desi    28.127198   5.252257  18.190172

=== Solusi Ideal ===
Ideal Positif : [28.127198 13.130643 18.190172]
Ideal Negatif : [11.250879  5.252257 10.914103]

=== Hasil Akhir TOPSIS ===
Alternatif       D+        D-  Nilai Preferensi  Ranking
    Taufan  7.276069 18.624690          0.719079        1
      Desi  7.878386 18.378011          0.699944        2
     Bunga 10.724278 16.876319          0.611448        3
      Yoga 16.876319 10.724278          0.388552        4
```

4.2 Mengubah C1 dan C2 adalah menjadi atribut Cost, kemudian menganlisis hasil dari program yang dibuat

```
# C1 = Cost
# C2 = Cost
# C3 = Benefit

ideal_positif = np.array([
    np.min(terbobot[:, 0]),  # C1 Cost
    np.min(terbobot[:, 1]),  # C2 Cost
    np.max(terbobot[:, 2])   # C3 Benefit
])

ideal_negatif = np.array([
    np.max(terbobot[:, 0]),  # C1 Cost
    np.max(terbobot[:, 1]),  # C2 Cost
    np.min(terbobot[:, 2])   # C3 Benefit
])

print("\n=== Solusi Ideal ===")
print("Ideal Positif :", np.round(ideal_positif, 6))
print("Ideal Negatif :", np.round(ideal_negatif, 6))

# 3. Menghitung jarak terhadap solusi ideal
D_plus = np.sqrt(np.sum((terbobot - ideal_positif)**2, axis=1))
D_minus = np.sqrt(np.sum((terbobot - ideal_negatif)**2, axis=1))

# 4. Menghitung nilai preferensi
V = D_minus / (D_plus + D_minus)
```

Output:

```
=== Matriks Keputusan ===
         C1   C2   C3
Taufan  5.0  5.0  3.0
Bunga   5.0  2.0  3.0
Yoga    2.0  5.0  5.0
Desi    5.0  2.0  5.0

=== Matriks Normalisasi Terbobot ===
               C1         C2         C3
Taufan  28.127198  13.130643  10.914103
Bunga   28.127198   5.252257  10.914103
Yoga    11.250879  13.130643  18.190172
Desi    28.127198   5.252257  18.190172

=== Solusi Ideal ===
Ideal Positif : [11.250879  5.252257 18.190172]
Ideal Negatif : [28.127198 13.130643 10.914103]

=== Hasil TOPSIS C1 dan C2 sebagai Cost ===
Alternatif        D+        D-  Nilai Preferensi  Ranking
      Yoga  7.878386 18.378011          0.699944        1
      Desi 16.876319 10.724278          0.388552        2
     Bunga 18.378011  7.878386          0.300056        3
    Taufan 19.995506  0.000000          0.000000        4
```

Penjelasan:

Pada bagian ini dilakukan perubahan jenis atribut pada C1 dan C2 menjadi atribut cost, sedangkan C3 tetap sebagai atribut benefit. Perubahan tersebut terutama memengaruhi proses penentuan solusi ideal positif dan solusi ideal negatif pada program TOPSIS.

Perubahan pada kode tersebut dilakukan karena aturan penentuan solusi ideal pada atribut cost berbeda dengan atribut benefit. Pada atribut cost, nilai yang lebih kecil dianggap lebih baik. Oleh karena itu, untuk solusi ideal positif pada C1 dan C2

digunakan fungsi np.min(), sedangkan untuk solusi ideal negatif digunakan fungsi np.max().

Sebaliknya, C3 tetap sebagai atribut benefit. Pada atribut benefit, nilai yang lebih besar dianggap lebih baik. Oleh karena itu, untuk C3 digunakan np.max() pada solusi ideal positif dan np.min() pada solusi ideal negatif. Aturan ini sesuai dengan konsep TOPSIS, yaitu solusi ideal positif menggunakan nilai terbaik dari setiap kriteria dan solusi ideal negatif menggunakan nilai terburuk

Secara matematis, penentuan solusi ideal dapat ditulis sebagai:

$$A^{+} = \begin{cases} \min(y_{ij}), & \text{jika kriteria cost} \\ \max(y_{ij}), & \text{jika kriteria benefit} \end{cases}$$

Sedangkan solusi ideal negatif:

$$A^{-} = \begin{cases} \max(y_{ij}), & \text{jika kriteria cost} \\ \min(y_{ij}), & \text{jika kriteria benefit} \end{cases}$$

Nilai tersebut menunjukkan bahwa untuk C1 dan C2 yang berjenis cost, nilai terkecil digunakan sebagai nilai ideal positif. Sementara itu, untuk C3 yang tetap benefit, nilai terbesar digunakan sebagai nilai ideal positif.

Setelah solusi ideal positif dan negatif berubah, perhitungan jarak setiap alternatif terhadap solusi ideal tetap menggunakan rumus Euclidean yang sama. Jadi, rumus jarak tidak berubah; yang berubah hanya nilai solusi ideal yang digunakan sebagai acuan. Modul juga menggunakan prinsip yang sama dalam perhitungan jarak alternatif terhadap solusi ideal positif dan negatif

Berdasarkan hasil tersebut, Yoga menjadi alternatif terbaik dengan nilai preferensi sebesar 0,699944. Hasil ini berbeda dengan kondisi awal ketika seluruh kriteria dianggap sebagai atribut benefit, di mana Taufan berada pada peringkat pertama. Perubahan ranking terjadi karena C1 dan C2 sekarang dinilai berdasarkan prinsip cost, sehingga nilai yang lebih kecil pada kedua kriteria tersebut dianggap lebih baik.

4.3 Menjelaskan baris-baris kode program pada soal nomer 1

Source Code	Penjelasan
`import numpy as np`	Mengimpor library **NumPy** dan memberikan alias np. NumPy digunakan untuk melakukan perhitungan numerik, operasi array,

	serta perhitungan matriks pada metode TOPSIS.
`import pandas as pd`	Mengimpor library **Pandas** dengan alias pd. Pandas digunakan untuk membuat dan menampilkan data dalam bentuk tabel atau DataFrame.
`# Data alternatif`	Komentar yang menunjukkan bahwa bagian berikutnya digunakan untuk mendefinisikan data alternatif.
`alternatif = ["Taufan", "Bunga", "Yoga", "Desi"]`	Membuat list yang berisi nama-nama alternatif yang akan dinilai menggunakan metode TOPSIS, yaitu Taufan, Bunga, Yoga, dan Desi.
`# Matriks keputusan`	Komentar yang menandai bagian pembuatan matriks keputusan.
`X = np.array([`	Membuat matriks keputusan menggunakan fungsi np.array(). Matriks ini akan menyimpan nilai setiap alternatif terhadap setiap kriteria.
`[5, 5, 3],`	Menentukan nilai alternatif **Taufan**, yaitu C1 = 5, C2 = 5, dan C3 = 3.
`[5, 2, 3],`	Menentukan nilai alternatif **Bunga**, yaitu C1 = 5, C2 = 2, dan C3 = 3.
`[2, 5, 5],`	Menentukan nilai alternatif **Yoga**, yaitu C1 = 2, C2 = 5, dan C3 = 5.
`[5, 2, 5]`	Menentukan nilai alternatif **Desi**, yaitu C1 = 5, C2 = 2, dan C3 = 5.
`], dtype=float)`	Mengakhiri pembuatan matriks keputusan dan menetapkan tipe data float agar nilai dapat digunakan dalam perhitungan desimal.
`# Bobot kriteria`	Komentar yang menandai bagian penentuan bobot setiap kriteria.
`bobot = np.array([50, 20, 30], dtype=float)`	Membuat array bobot kriteria, yaitu C1 = 50, C2 = 20, dan C3 = 30. Tipe data dibuat float agar dapat digunakan dalam proses perkalian matriks.
`# Menampilkan matriks keputusan`	Komentar yang menunjukkan bahwa bagian berikut digunakan untuk menampilkan matriks keputusan.

`df_x = pd.DataFrame(`	Membuat tabel DataFrame dari matriks keputusan agar hasil lebih mudah dibaca.
`X,`	Menggunakan matriks keputusan X sebagai data utama dalam DataFrame.
`index=alternatif,`	Menggunakan daftar nama alternatif sebagai indeks atau nama baris tabel.
`columns=["C1", "C2", "C3"]`	Memberikan nama kolom C1, C2, dan C3 sesuai dengan kriteria yang digunakan.
`)`	Mengakhiri pembuatan DataFrame matriks keputusan.
`print("=== Matriks Keputusan ===")`	Menampilkan judul **Matriks Keputusan** pada output program.
`print(df_x)`	Menampilkan tabel matriks keputusan yang telah dibuat.
`# 1. Normalisasi matriks keputusan`	Komentar yang menandai tahap pertama perhitungan TOPSIS, yaitu normalisasi matriks keputusan.
`normalisasi = X / np.sqrt(np.sum(X**2, axis=0))`	Menghitung matriks keputusan ternormalisasi. X**2 mengkuadratkan setiap nilai, np.sum(..., axis=0) menjumlahkan nilai kuadrat berdasarkan kolom atau kriteria, np.sqrt() menghitung akar kuadratnya, kemudian setiap nilai pada X dibagi dengan hasil tersebut.
`df_normalisasi = pd.DataFrame(`	Membuat DataFrame untuk menampilkan hasil matriks normalisasi.
`normalisasi,`	Menggunakan hasil perhitungan normalisasi sebagai data tabel.
`index=alternatif,`	Menggunakan nama alternatif sebagai nama baris.
`columns=["C1", "C2", "C3"]`	Menentukan nama kolom berdasarkan kriteria C1, C2, dan C3.
`)`	Mengakhiri pembuatan DataFrame normalisasi.
`print("\n=== Matriks Keputusan Ternormalisasi ===")`	Menampilkan judul matriks keputusan ternormalisasi. \n digunakan untuk memberikan baris kosong sebelum teks ditampilkan.
`print(df_normalisasi.round(6))`	Menampilkan matriks normalisasi dengan pembulatan hingga 6 angka di belakang koma.

`# 2. Matriks normalisasi terbobot`	Komentar yang menandai tahap kedua, yaitu menghitung matriks normalisasi terbobot.
`terbobot = normalisasi * bobot`	Mengalikan setiap nilai hasil normalisasi dengan bobot kriteria yang bersesuaian.
`df_terbobot = pd.DataFrame(`	Membuat tabel DataFrame dari matriks normalisasi terbobot.
`terbobot,`	Menggunakan hasil perhitungan terbobot sebagai isi tabel.
`index=alternatif,`	Menggunakan nama alternatif sebagai nama baris.
`columns=["C1", "C2", "C3"]`	Menentukan nama kolom C1, C2, dan C3.
`)`	Mengakhiri pembuatan DataFrame matriks terbobot.
`print("\n=== Matriks Normalisasi Terbobot ===")`	Menampilkan judul matriks normalisasi terbobot pada output.
`print(df_terbobot.round(6))`	Menampilkan matriks normalisasi terbobot dengan pembulatan 6 angka di belakang koma.
`# 3. Solusi ideal positif dan negatif`	Komentar yang menandai tahap penentuan solusi ideal positif dan solusi ideal negatif.
`# Semua kriteria merupakan Benefit`	Menjelaskan bahwa pada program pertama semua kriteria dianggap sebagai atribut **benefit**, sehingga nilai terbesar menjadi solusi ideal positif dan nilai terkecil menjadi solusi ideal negatif.
`ideal_positif = np.max(terbobot, axis=0)`	Menentukan solusi ideal positif dengan mengambil nilai maksimum dari setiap kolom pada matriks normalisasi terbobot.
`ideal_negatif = np.min(terbobot, axis=0)`	Menentukan solusi ideal negatif dengan mengambil nilai minimum dari setiap kolom pada matriks normalisasi terbobot.
`print("\n=== Solusi Ideal ===")`	Menampilkan judul bagian solusi ideal pada output.
`print("Ideal Positif :", np.round(ideal_positif, 6))`	Menampilkan nilai solusi ideal positif dan membulatkannya hingga 6 angka di belakang koma.
`print("Ideal Negatif :", np.round(ideal_negatif, 6))`	Menampilkan nilai solusi ideal negatif dan membulatkannya hingga 6 angka di belakang koma.
`# 4. Menghitung jarak terhadap solusi ideal`	Komentar yang menandai tahap perhitungan jarak masing-masing alternatif terhadap solusi ideal.

`D_plus = np.sqrt(np.sum((terbobot - ideal_positif)**2, axis=1))`	Menghitung jarak setiap alternatif terhadap solusi ideal positif menggunakan jarak Euclidean. Selisih antara matriks terbobot dan solusi ideal positif dikuadratkan, dijumlahkan per baris, kemudian diakarkan.
`D_minus = np.sqrt(np.sum((terbobot - ideal_negatif)**2, axis=1))`	Menghitung jarak setiap alternatif terhadap solusi ideal negatif menggunakan cara yang sama, tetapi menggunakan solusi ideal negatif sebagai pembanding.
`# 5. Menghitung nilai preferensi`	Komentar yang menandai tahap perhitungan nilai preferensi akhir TOPSIS.
`V = D_minus / (D_plus + D_minus)`	Menghitung nilai preferensi setiap alternatif. Nilai ini diperoleh dari jarak terhadap solusi ideal negatif dibagi dengan jumlah jarak terhadap solusi ideal positif dan negatif.
`# Menampilkan hasil`	Komentar yang menunjukkan bahwa bagian berikut digunakan untuk menyusun hasil akhir dalam bentuk tabel.
`hasil = pd.DataFrame({`	Membuat DataFrame baru untuk menampung hasil akhir TOPSIS.
`"Alternatif": alternatif,`	Membuat kolom Alternatif yang berisi nama setiap alternatif.
`"D+": D_plus,`	Membuat kolom D+ yang berisi jarak setiap alternatif terhadap solusi ideal positif.
`"D-": D_minus,`	Membuat kolom D- yang berisi jarak setiap alternatif terhadap solusi ideal negatif.
`"Nilai Preferensi": V`	Membuat kolom Nilai Preferensi yang berisi nilai akhir preferensi masing-masing alternatif.
`})`	Mengakhiri pembuatan DataFrame hasil.
`hasil["Ranking"] = hasil["Nilai Preferensi"].rank(`	Membuat kolom baru bernama Ranking berdasarkan nilai pada kolom Nilai Preferensi.
`ascending=False,`	Menentukan bahwa proses ranking dilakukan dari nilai terbesar ke nilai terkecil. Nilai preferensi terbesar akan memperoleh ranking pertama.
`method="min"`	Menentukan metode ranking jika terdapat nilai yang sama. Alternatif dengan nilai yang sama akan

	memperoleh ranking terkecil yang sama.
`).astype(int)`	Mengubah hasil ranking menjadi tipe bilangan bulat atau integer.
`hasil = hasil.sort_values("Ranking")`	Mengurutkan tabel hasil berdasarkan kolom Ranking, mulai dari peringkat pertama hingga peringkat terakhir.
`print("\n=== Hasil Akhir TOPSIS ===")`	Menampilkan judul hasil akhir TOPSIS pada output program.
`print(hasil.round(6).to_string(index=False))`	Menampilkan hasil akhir TOPSIS dengan pembulatan 6 angka di belakang koma dan tanpa menampilkan indeks bawaan dari Pandas.

4.4 Kelebihan dan kekurangan Metode TOPSIS

Kelebihan Metode TOPSIS

- Konsep perhitungannya relatif sederhana dan mudah dipahami. TOPSIS menggunakan konsep jarak terhadap solusi ideal positif dan solusi ideal negatif, sehingga alur perhitungannya cukup sistematis.
- Dapat digunakan untuk banyak alternatif dan kriteria. Metode TOPSIS cocok digunakan pada permasalahan pengambilan keputusan yang melibatkan beberapa alternatif dengan beberapa kriteria penilaian.
- Dapat menangani atribut benefit dan cost. Kriteria benefit maupun cost dapat digunakan secara bersamaan dengan menyesuaikan penentuan solusi ideal positif dan solusi ideal negatif.
- Mempertimbangkan kondisi terbaik dan terburuk sekaligus. Alternatif yang baik tidak hanya dilihat dari kedekatannya dengan solusi ideal positif, tetapi juga dari seberapa jauh alternatif tersebut dari solusi ideal negatif.
- Menghasilkan nilai preferensi yang memudahkan proses perangkingan. Nilai preferensi akhir dapat digunakan untuk mengurutkan alternatif dari yang paling direkomendasikan hingga yang paling rendah.
- Dapat diterapkan dengan mudah menggunakan Excel maupun program. Tahapan perhitungan TOPSIS dapat diimplementasikan menggunakan Microsoft Excel maupun bahasa pemrograman seperti Python.

Kekurangan Metode TOPSIS

- Hasil sangat dipengaruhi oleh bobot kriteria.
  Jika bobot yang diberikan kurang tepat, maka hasil perangkingan juga dapat berubah dan menjadi kurang representatif.
- Tidak memiliki metode khusus untuk menentukan bobot.
  TOPSIS hanya menggunakan bobot yang telah ditentukan sebelumnya, sehingga proses penentuan bobot biasanya membutuhkan metode lain atau pertimbangan dari pengambil keputusan.
- Sensitif terhadap perubahan nilai alternatif.
  Perubahan nilai pada matriks keputusan dapat memengaruhi hasil normalisasi, solusi ideal, hingga urutan ranking.
- Menganggap setiap kriteria berdiri sendiri.
  TOPSIS tidak secara langsung mempertimbangkan hubungan atau ketergantungan antar-kriteria.
- Hasil akhir bergantung pada proses normalisasi.
  Karena nilai awal harus dinormalisasi terlebih dahulu, hasil akhir dapat dipengaruhi oleh metode normalisasi yang digunakan.
- Semakin banyak alternatif dan kriteria, proses perhitungan semakin kompleks jika dilakukan secara manual.

5. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, metode Technique for Order Preference by Similarity to Ideal Solution (TOPSIS) dapat digunakan untuk membantu proses pengambilan keputusan dengan membandingkan beberapa alternatif berdasarkan sejumlah kriteria. Tahapan perhitungannya meliputi pembentukan matriks keputusan, normalisasi, pembobotan, penentuan solusi ideal positif dan negatif, perhitungan jarak setiap alternatif terhadap solusi ideal, hingga penentuan nilai preferensi dan perangkingan. Melalui tahapan tersebut, setiap alternatif dapat dibandingkan secara sistematis berdasarkan tingkat kedekatannya dengan kondisi ideal.

Pada studi kasus praktikum, seluruh kriteria awal diperlakukan sebagai atribut benefit. Hasil perhitungan menunjukkan bahwa Taufan memperoleh nilai preferensi tertinggi sebesar 0,719079, sehingga menempati peringkat pertama dan menjadi alternatif yang paling direkomendasikan. Hasil ini menunjukkan bahwa Taufan memiliki jarak yang

relatif dekat terhadap solusi ideal positif dan cukup jauh dari solusi ideal negatif dibandingkan alternatif lainnya.

Setelah C1 dan C2 diubah menjadi atribut cost, hasil perangkingan mengalami perubahan. Yoga memperoleh nilai preferensi tertinggi sebesar 0,699944 dan menjadi alternatif terbaik pada skenario tersebut. Hal ini membuktikan bahwa jenis atribut benefit dan cost memiliki pengaruh penting terhadap penentuan solusi ideal dan hasil akhir perangkingan. Dengan demikian, TOPSIS mampu memberikan hasil keputusan yang terstruktur, jelas, dan dapat disesuaikan dengan karakteristik masing-masing kriteria.